

Chương này trình bày quá trình phân tích thiết kế, hiện thực và đánh giá hệ thống đặt vé và quản lý sự kiện được phát triển trong đề án. Trước hết, chương làm rõ kiến trúc phần mềm được lựa chọn và cách áp dụng kiến trúc đó vào hệ thống thực tế. Tiếp theo, chương mô tả thiết kế tổng quan, thiết kế chi tiết, quá trình xây dựng ứng dụng, kết quả đạt được, kiểm thử và định hướng triển khai. Các nội dung trong chương này đóng vai trò kết nối giữa phần khảo sát yêu cầu ở các chương trước với phần đóng góp và tổng kết ở các chương sau, qua đó cho thấy hệ thống đã được tổ chức và hiện thực hóa như thế nào trên cả phía ứng dụng di động lẫn phía máy chủ.

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

Đối với bài toán đặt vé và quản lý sự kiện trên thiết bị di động, hệ thống cần đồng thời hỗ trợ nhiều chức năng khác nhau như quản lý tài khoản, tìm kiếm sự kiện, quản lý sự kiện, đặt vé, thanh toán, nhắn tin, thông báo và check-in bằng mã mã phản hồi nhanh (Quick Response - QR). Các chức năng này có liên hệ chặt chẽ với nhau, dùng chung dữ liệu và cùng phục vụ một luồng nghiệp vụ xuyên suốt từ lúc người dùng khám phá sự kiện cho tới lúc tham dự thực tế. Vì vậy, kiến trúc phù hợp nhất cho hệ thống trong phạm vi đề án là kiến trúc client-server theo hướng mobile-first, kết hợp với tổ chức phần mềm theo mô hình ba lớp ở phía máy chủ. Theo cách hiểu này, ứng dụng di động đóng vai trò lớp giao diện và tương tác với người dùng, máy chủ backend đóng vai trò lớp xử lý nghiệp vụ, còn cơ sở dữ liệu là lớp lưu trữ dữ liệu tập trung.

Kiến trúc ba lớp được lựa chọn thay cho các hướng như microservice hay backendless vì nó phù hợp hơn với quy mô hiện tại của hệ thống. Nếu sử dụng microservice, mỗi nhóm chức năng như tài khoản, sự kiện, vé, thông báo hay chat có thể được tách thành các dịch vụ độc lập. Tuy nhiên, với phạm vi của đề án, cách tiếp cận đó làm tăng độ phức tạp triển khai, quản lý giao tiếp giữa các dịch vụ và đồng bộ dữ liệu, trong khi lượng người dùng và số lượng giao dịch chưa ở mức cần phân tán mạnh. Ngược lại, kiến trúc đơn khối có tổ chức theo lớp giúp hệ thống dễ xây dựng, dễ bảo trì hơn trong giai đoạn phát triển đầu tiên, đồng thời vẫn đủ rõ ràng để mở rộng khi cần. So với mô hình backendless, việc xây dựng backend riêng còn cho phép kiểm soát chặt chẽ hơn các nghiệp vụ đặc thù như quản lý nhiều hạng vé, kiểm soát trạng thái thanh toán, gửi lời mời, kiểm tra vé đã dùng hay chưa và xác nhận check-in theo sự kiện cụ thể.

Xét trên hệ thống cụ thể của đề tài, lớp giao diện và tương tác được hiện thực bằng ứng dụng di động React Native kết hợp Expo. Thành phần này bao gồm các

màn hình chức năng trong thư mục `frontend/pages`, các thành phần giao diện dùng lại trong `frontend/components`, hệ điều hướng trong `frontend/-navigators`, cùng các ngữ cảnh quản lý trạng thái như xác thực và bản địa hóa trong `frontend/context`. Nhiệm vụ của lớp này là tiếp nhận thao tác của người dùng, tổ chức luồng di chuyển giữa các màn hình, hiển thị dữ liệu và gửi yêu cầu đến backend thông qua các service như `auth.service`, `event.service`, `ticket.service`, `notification.service` hay `chat.service`. Nói cách khác, đây là phần gần nhất với người dùng cuối và chịu trách nhiệm thể hiện toàn bộ chức năng của hệ thống trên thiết bị di động.

Lớp xử lý nghiệp vụ được tổ chức ở phía backend theo một dạng MVC điều chỉnh cho ứng dụng API và dịch vụ thời gian thực. Trong hệ thống này, vai trò tương đương với phần Model được thể hiện bởi các schema và model Mongoose trong thư mục `backend/src/models`, nơi quản lý cấu trúc dữ liệu của người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và message. Vai trò tương đương với phần Controller là các controller trong `backend/src/controllers`, phụ trách xử lý từng luồng nghiệp vụ như đăng nhập, tạo sự kiện, đặt vé, xác nhận thanh toán, gửi lời mời, nhắn tin hay check-in. Bổ sung cho đó là các middleware và service trong `backend/src/middleware`, `backend/src/services` và `backend/src/utils`, dùng để giải quyết những nghiệp vụ dùng chung như xác thực JWT, gửi email, gửi thông báo đẩy, sinh mã QR, gọi ý nội dung sự kiện bằng AI hoặc lập lịch nhắc sự kiện. Phần View trong mô hình MVC truyền thống không được đặt ở phía backend như các ứng dụng web server-rendered, mà được chuyển sang ứng dụng di động ở frontend. Đây là điểm điều chỉnh quan trọng để phù hợp với kiến trúc ứng dụng di động hiện đại.

Ở mức tổ chức truy cập, các route trong `backend/src/routes` đóng vai trò cổng vào của lớp dịch vụ. Mỗi nhóm route như `/api/auth`, `/api/users`, `/api/events`, `/api/tickets`, `/api/notifications` hay `/api/chat` tiếp nhận yêu cầu từ ứng dụng khách, sau đó chuyển tiếp cho controller tương ứng. Điều này tạo nên một chuỗi xử lý tương đối rõ ràng: người dùng thao tác trên giao diện di động, service ở frontend gửi yêu cầu HTTP hoặc sự kiện socket tới backend, route tiếp nhận yêu cầu, controller xử lý nghiệp vụ, model truy cập dữ liệu, sau đó kết quả được trả ngược về giao diện. Với các chức năng gần thời gian thực như chat, hệ thống mở rộng thêm một kênh giao tiếp bằng Socket.IO ở tệp `backend/src/socket.ts`. Đây có thể xem là một thành phần bổ sung cho lớp xử lý nghiệp vụ, giúp hệ thống vượt ra ngoài mô hình request-response thuần túy mà vẫn không phá vỡ cấu trúc tổng thể.

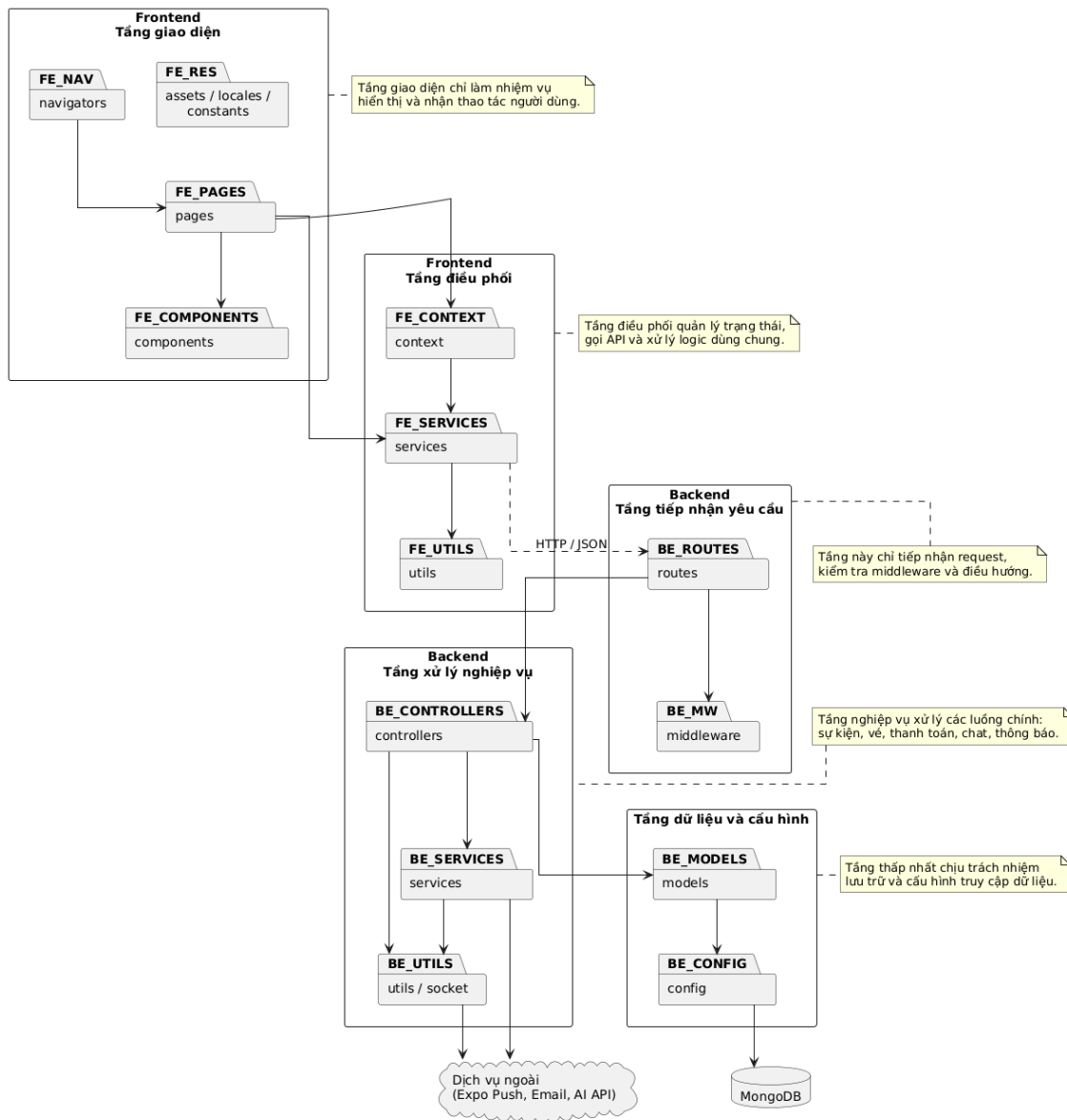
Lớp dữ liệu của hệ thống được xây dựng trên MongoDB và được truy cập thông

qua Mongoose. Việc dùng cơ sở dữ liệu tài liệu giúp hệ thống linh hoạt trong việc biểu diễn các thực thể có cấu trúc thay đổi theo nghiệp vụ, chẳng hạn sự kiện có thể có nhiều hạng vé, trạng thái duyệt, thông tin tọa độ hoặc dữ liệu liên quan đến người tổ chức. Với bài toán của đề tài, dữ liệu không chỉ cần được lưu trữ mà còn phải phản ánh quan hệ giữa nhiều thực thể như người dùng – sự kiện – vé – thanh toán – thông báo. Bởi vậy, lớp dữ liệu không chỉ đóng vai trò lưu trữ mà còn là nền tảng để bảo đảm tính nhất quán cho các thao tác nghiệp vụ quan trọng như tăng số lượng vé đã bán, đánh dấu vé đã check-in hoặc ghi nhận lời mời đã gửi.

Từ góc nhìn tổng thể, kiến trúc được áp dụng cho hệ thống có thể mô tả là kiến trúc client-server ba lớp, trong đó backend được tổ chức theo tinh thần MVC điều chỉnh cho ứng dụng API. Đây là lựa chọn phù hợp với sản phẩm của đề án vì vừa bảo đảm tính mạch lạc trong thiết kế, vừa đủ linh hoạt để tích hợp thêm các thành phần mở rộng như thông báo đẩy, chat thời gian thực, AI hỗ trợ nội dung và kiểm soát tham dự bằng mã QR. Quan trọng hơn, kiến trúc này cho phép nhóm chức năng của hệ thống được phân chia tương đối rõ ràng, giúp cho việc phát triển, bảo trì và mở rộng về sau thuận lợi hơn so với một cấu trúc mã nguồn rời rạc hoặc quá phức tạp so với quy mô hiện tại của đề tài.

0.1.2 Thiết kế tổng quan

Biểu đồ gói tổng quan của hệ thống được xây dựng theo hướng phân tầng rõ ràng giữa phía ứng dụng di động và phía máy chủ. Mục tiêu của cách tổ chức này là giúp các thành phần đảm nhiệm đúng trách nhiệm của mình, hạn chế sự phụ thuộc chéo và thuận lợi cho việc bảo trì sau này. Ở mức khái quát, hệ thống được chia thành các nhóm gói chính gồm: nhóm giao diện người dùng ở frontend, nhóm điều phối và truy cập dịch vụ ở frontend, nhóm tiếp nhận yêu cầu ở backend, nhóm xử lý nghiệp vụ và tích hợp ở backend, cùng với nhóm dữ liệu và cấu hình ở tầng thấp nhất.



Hình 0.1: Biểu đồ gói tổng quan của hệ thống event-booking-app

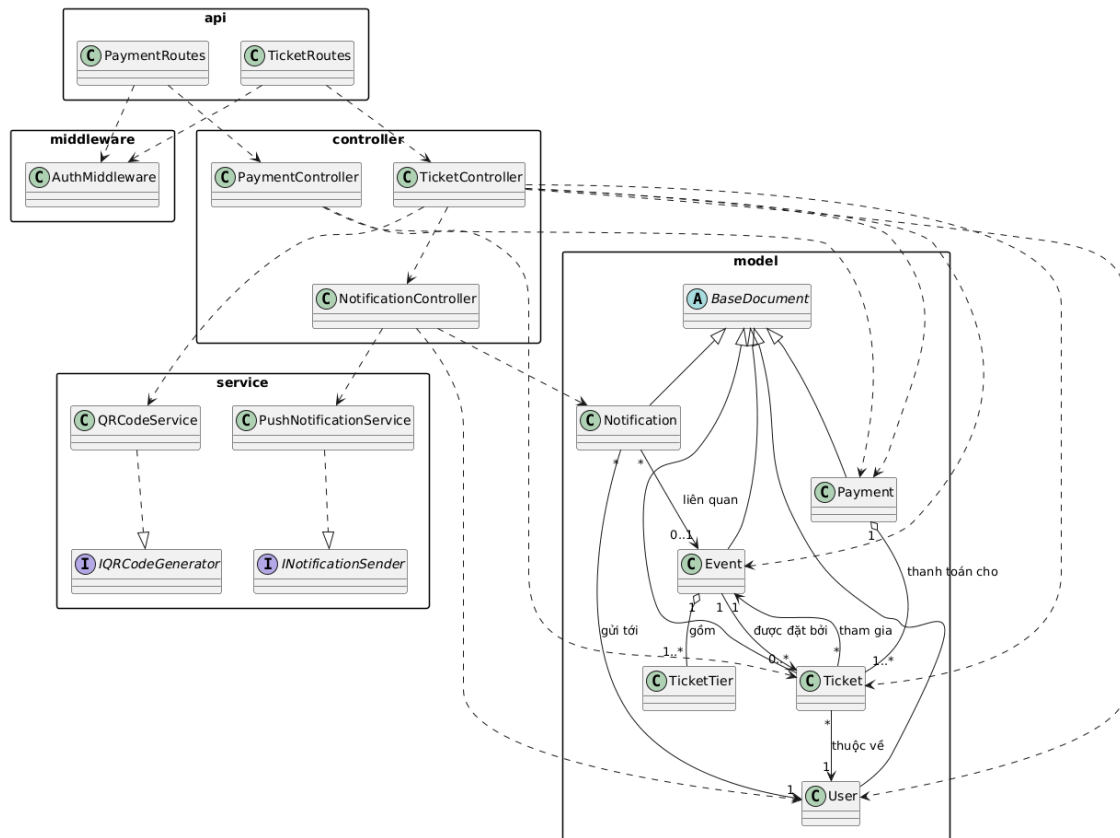
Ở phía frontend, các package có thể được chia thành hai tầng chính. Tầng giao diện bao gồm pages, components, navigators, assets, locales và constants. Trong đó, pages chứa các màn hình nghiệp vụ cụ thể như đăng ký, đặt vé, quản lý sự kiện, chat và thông báo; components chứa các thành phần giao diện dùng lại; navigators chịu trách nhiệm tổ chức luồng điều hướng; còn constants, assets và locales cung cấp các tài nguyên dùng chung. Tầng điều phối ở frontend bao gồm context, services và utils. Package context lưu trạng thái dùng chung như thông tin xác thực và ngôn ngữ, package services thực hiện giao tiếp với backend thông qua API, còn utils xử lý các hàm phụ trợ. Cấu trúc này bảo đảm giao diện không truy cập trực tiếp tới backend hay dữ liệu lưu trữ, mà luôn đi qua tầng service và context tương ứng.

Ở phía backend, package routes là lớp tiếp nhận yêu cầu từ bên ngoài và phân

phối yêu cầu đến các controller tương ứng. Package `middleware` cung cấp các cơ chế kiểm tra xác thực, phân quyền và xử lý lỗi dùng chung cho toàn hệ thống. Package `controllers` là trung tâm điều phối nghiệp vụ, nơi hiện thực các luồng như đăng nhập, tạo sự kiện, đặt vé, gửi lời mời, check-in và xử lý thông báo. Package `services` và `utils` bổ sung các chức năng hỗ trợ hoặc tích hợp như sinh gợi ý nội dung sự kiện bằng AI, gửi email, gửi thông báo đẩy, lập lịch nhắc sự kiện, sinh mã QR và xử lý socket thời gian thực. Package `models` đại diện cho tầng dữ liệu nghiệp vụ, mô tả các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và tin nhắn. Cuối cùng, package `config` chịu trách nhiệm kết nối cơ sở dữ liệu và nạp biến môi trường, tạo nền tảng kỹ thuật cho toàn bộ các package phía trên.

0.1.3 Thiết kế chi tiết gói

Để làm rõ cách tổ chức các lớp trong hệ thống, đề tài lựa chọn nhóm package giải quyết nghiệp vụ đặt vé, thanh toán và tham dự sự kiện để mô tả chi tiết. Đây là nhóm package quan trọng vì nó kết nối trực tiếp nhiều thực thể nghiệp vụ nhất của hệ thống, đồng thời bao trùm các thao tác cốt lõi như chọn vé, xử lý thanh toán, sinh vé điện tử, gửi thông báo và check-in bằng mã QR. Ở mức thiết kế gói, biểu đồ tập trung vào năm nhóm thành phần chính là `api`, `middleware`, `controller`, `service` và `model`. Mỗi nhóm được phân công một vai trò rõ ràng nhằm tránh chồng chéo trách nhiệm và giúp luồng xử lý nghiệp vụ có thể theo dõi được từ đầu tới cuối.



Hình 0.2: Biểu đồ thiết kế gói cho nghiệp vụ đặt vé, thanh toán và tham dự sự kiện

Trong thiết kế này, package `api` gồm các lớp như `TicketRoutes` và `PaymentRoutes`, chịu trách nhiệm tiếp nhận yêu cầu từ phía ứng dụng khách và chuyển yêu cầu sang tầng điều phối. Package `middleware` chứa lớp `AuthMiddleware`, được dùng để kiểm tra token xác thực và bảo vệ các chức năng chỉ dành cho người dùng đã đăng nhập. Package `controller` bao gồm các lớp như `TicketController`, `PaymentController` và `NotificationController`. Đây là nơi tổ chức luồng nghiệp vụ chính, chẳng hạn kiểm tra loại vé, khởi tạo giao dịch, xác nhận thanh toán, gọi dịch vụ sinh mã QR, cập nhật trạng thái vé và gửi thông báo sau giao dịch. Package `service` bao gồm các dịch vụ hỗ trợ như `QRCodeService` và `PushNotificationService`, cùng với các interface tương ứng như `IQRCodeGenerator` và `INotificationSender`. Việc đưa interface vào thiết kế giúp tách biệt giữa phần hợp đồng chức năng và phần hiện thực cụ thể, thuận lợi cho việc thay thế hoặc mở rộng dịch vụ về sau. Package `model` bao gồm các lớp dữ liệu chính như `Event`, `Ticket`, `Payment`, `Notification`, `User` và `TicketTier`, cùng lớp cơ sở trừu tượng `BaseDocument`.

Các quan hệ giữa các lớp trong biểu đồ gói thể hiện rõ cách vận hành của hệ thống. Quan hệ phụ thuộc (dependency) xuất hiện khi các lớp route sử dụng mid-

dleware và controller, hoặc khi controller cần gọi service và truy cập model để xử lý nghiệp vụ. Quan hệ thực thi (implementation) xuất hiện ở chỗ `QRCodeService` hiện thực `IQRCodeGenerator` và `PushNotificationService` hiện thực `INotificationSender`. Quan hệ kế thừa (inheritance) được dùng để mô tả việc các model nghiệp vụ kế thừa từ lớp cơ sở `BaseDocument`. Quan hệ kết tập (aggregation) có thể thấy trong trường hợp `Payment` tập hợp nhiều đối tượng `Ticket` thuộc cùng một giao dịch, trong khi quan hệ hợp thành (composition) thể hiện ở việc `Event` bao gồm nhiều `TicketTier` như một phần cấu thành của sự kiện. Ngoài ra, quan hệ kết hợp (association) được dùng để phản ánh các mối liên hệ nghiệp vụ như mỗi `Ticket` gắn với một `User` và một `Event`, hoặc một `Notification` được gửi tới một `User` và có thể liên quan tới một `Event` cụ thể.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Hệ thống được định hướng là một ứng dụng mobile-first, do đó thiết kế giao diện tập trung chủ yếu vào môi trường điện thoại thông minh và các thiết bị di động cầm tay. Về mặt mục tiêu hiển thị, ứng dụng hướng tới các màn hình có độ phân giải phổ biến trên smartphone hiện nay, hỗ trợ tốt ở cả các thiết bị cỡ nhỏ, thiết bị kích thước trung bình và máy tính bảng. Trong mã nguồn hiện tại, nhiều màn hình như trang chủ, trang chi tiết sự kiện, màn hình chọn vé hay màn hình chat đều sử dụng cơ chế `useWindowDimensions` để điều chỉnh giao diện theo chiều rộng thực tế của thiết bị. Điều này cho thấy thiết kế giao diện của hệ thống không cố định ở một kích thước duy nhất, mà hướng tới khả năng thích nghi với nhiều khung hiển thị. Trên thiết bị nhỏ, khoảng cách lề, chiều cao ô nhập liệu, kích thước biểu tượng và cỡ chữ được thu gọn để bảo đảm không tràn bố cục; trong khi với máy tính bảng, các thành phần được mở rộng hợp lý để tận dụng không gian hiển thị.

Về số lượng màu sắc và định hướng hiển thị, hệ thống sử dụng môi trường giao diện đồ họa tiêu chuẩn của thiết bị di động với dải màu đầy đủ, hỗ trợ hiển thị văn bản, ảnh, icon, nền khối, gradient và phản hồi trạng thái bằng màu. Trong mã nguồn, bảng màu của ứng dụng được chuẩn hóa tập trung trong các tệp `frontend/constants/colors.ts` và `frontend/constants/theme.ts`. Màu chủ đạo của hệ thống là cam `#F76B10`, được dùng để nhấn mạnh các thao tác chính như đặt vé, xác nhận hoặc điều hướng tới các chức năng nổi bật. Bên cạnh đó, hệ thống sử dụng thêm các màu phụ như xanh dương, vàng, xanh lá, trắng, xám và các màu trạng thái để biểu diễn thông tin khác nhau. Cách tổ chức này giúp giao diện có tính thống nhất cao và hỗ trợ việc thay đổi chủ đề hoặc mở rộng sang dark mode về sau.

Nguyên tắc tổ chức giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng ưu tiên khả năng thao tác nhanh, dễ hiểu và phù hợp với hành vi sử dụng trên điện thoại. Các màn hình nghiệp vụ chính đều tuân theo cấu trúc ba phần: vùng điều hướng phía trên, vùng nội dung trung tâm và vùng thao tác chính ở cuối màn hình hoặc tại những vị trí dễ chạm. Ví dụ, trang chủ gồm phần tiêu đề tài khoản và vị trí người dùng, thanh tìm kiếm, các nhóm danh mục và danh sách sự kiện; màn hình chi tiết sự kiện tổ chức nội dung theo trình tự banner – thông tin sự kiện – thao tác chính; còn màn hình mua vé và thanh toán được thiết kế theo luồng nhập liệu từng bước. Việc phân chia như vậy giúp người dùng nhanh chóng nhận biết phần nào dùng để xem thông tin, phần nào dùng để thao tác và phần nào là phản hồi của hệ thống.

Các thành phần giao diện cũng được chuẩn hóa theo một số quy tắc nhất quán. Nút thao tác chính thường dùng nền đậm hoặc màu nhấn, bo góc lớn và chữ đậm để tạo sự nổi bật. Các ô nhập liệu sử dụng nền sáng, viền rõ ràng, khoảng đệm tương đối lớn để dễ nhập trên màn hình cảm ứng. Các danh sách, thẻ sự kiện và vùng thông tin dùng bo góc mềm để giữ tính nhất quán với tổng thể thiết kế hiện đại của ứng dụng. Với các chức năng có tính hệ quả trực tiếp như thanh toán, xóa hoặc từ chối sự kiện, giao diện luôn cố gắng cung cấp dấu hiệu nhận biết rõ ràng thông qua màu sắc, vị trí nút và thông báo phản hồi. Hệ thống cũng sử dụng icon phổ biến như menu, tìm kiếm, vị trí, tin nhắn, vé và các trạng thái tăng giảm số lượng để giảm khối lượng văn bản phải hiển thị trên màn hình.

Chuẩn hóa vị trí thông điệp và phản hồi hệ thống

Một yêu cầu quan trọng của ứng dụng di động là thông điệp phản hồi phải xuất hiện đúng lúc và đủ rõ để người dùng không bị mất phương hướng khi thao tác. Trong thiết kế hiện tại, thông điệp phản hồi được tổ chức theo ba mức. Mức thứ nhất là phản hồi cục bộ ngay trong thành phần giao diện, ví dụ trạng thái nút bị mờ khi không thể bấm, số lượng vé không thể tăng vượt quota hoặc biểu tượng thay đổi theo trạng thái đã lưu. Mức thứ hai là phản hồi qua các hộp thoại cảnh báo hoặc thông báo ngắn khi thao tác thành công hay thất bại, chẳng hạn khi thêm thẻ, duyệt sự kiện hoặc gặp lỗi kết nối. Mức thứ ba là phản hồi ở mức hệ thống thông qua notification và push notification, áp dụng cho các tình huống như lời mời sự kiện, nhắc lịch, thanh toán thành công hoặc có tin nhắn mới. Việc phân tách phản hồi như vậy giúp cho người dùng luôn nhận được thông tin phù hợp với mức độ quan trọng của thao tác.

Định hướng thiết kế cho các màn hình quan trọng

Đối với màn hình trang chủ, định hướng thiết kế là ưu tiên khám phá nhanh. Người dùng cần nhìn thấy ngay thông tin cá nhân tối thiểu, thanh tìm kiếm, danh mục sự kiện và các sự kiện nổi bật. Vì vậy, bố cục được tổ chức theo chiều dọc, dùng thẻ sự kiện và các thanh danh mục nằm ngang để giảm thao tác cuộn không cần thiết. Đối với màn hình chi tiết sự kiện, thiết kế hướng tới việc cung cấp thông tin đầy đủ nhưng không gây quá tải, nên banner hình ảnh được đặt ở đầu trang, sau đó là khối thông tin tóm tắt, mô tả, thông tin người tổ chức và các hành động như đặt vé, nhấn tin hoặc lưu sự kiện. Đối với màn hình mua vé, thiết kế cần làm nổi bật việc lựa chọn hạng vé, số lượng và tổng tiền để người dùng dễ ra quyết định. Còn đối với màn hình chat, thiết kế được tổ chức theo mẫu hội thoại quen thuộc với khung tin nhắn, thời gian gửi, ô nhập liệu và phản hồi đọc tin nhằm giảm thời gian làm quen của người sử dụng.

Nhận xét chung về thiết kế giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng thực dụng, ưu tiên tính dễ dùng và sự nhất quán hơn là những hiệu ứng trình diễn phức tạp. Điểm mạnh của thiết kế nằm ở việc chuẩn hóa tương đối rõ ràng màu, kiểu chữ, vùng nhập liệu, biểu tượng và phản hồi hệ thống, đồng thời có xem xét tới khả năng thích nghi với nhiều kích thước màn hình khác nhau. Dù đây mới là một sản phẩm ở mức thử nghiệm, định hướng giao diện hiện tại đã đủ để làm nền tảng cho các luồng nghiệp vụ chính như khám phá sự kiện, đặt vé, thanh toán, quản lý vé và tương tác giữa các bên. Trong các bước tiếp theo, phần minh họa bằng ảnh giao diện thực tế có thể tiếp tục được bổ sung để làm rõ hơn cách những nguyên tắc thiết kế này được hiện thực hóa trên từng màn hình cụ thể.

Hình ảnh Wireframe 1 số giao diện:

Tạo sự kiện

Ảnh đại diện
Chon ảnh

Tiêu đề sự kiện
Workshop Mobile Event App

Danh mục
Music

Ngày tổ chức
2026-07-15

Thời gian
19:30

Địa điểm
Hà Nội

Mô tả
Mô tả ngắn về sự kiện...

Hạng vé

Hạng vé	Giá vé	Quota	
Standard	100000	100	
VIP	250000	30	

+ Thêm hạng vé

Gợi ý nội dung bằng AI | Lưu nháp | Gửi duyệt

Hình 0.3: Wireframe minh họa giao diện chức năng tạo sự kiện

Đặt vé

Sự kiện: Mobile Event Booking Workshop

Địa điểm: Hà Nội

Thời gian: 19:30 - 15/07/2026

Loại vé	Giá	Còn lại	Chọn
Standard	100000	100	⊗
VIP	250000	30	○

Số lượng: - 2 +

Thành tiền: 200000 VNĐ

Phương thức thanh toán: Wallet

Mã giảm giá

Xác nhận thanh toán

Hình 0.4: Wireframe minh họa giao diện chức năng đặt vé

0.2.2 Thiết kế lớp

Trong hệ thống đặt vé và quản lý sự kiện của đồ án, có nhiều lớp dữ liệu và lớp xử lý cùng phối hợp để thực hiện các nghiệp vụ. Tuy nhiên, nếu xét theo mức độ trung tâm đối với luồng nghiệp vụ chính, bốn lớp quan trọng nhất là `User`, `Event`, `Ticket` và `Payment`. Đây là các lớp phản ánh trực tiếp ba thực thể cốt lõi của hệ thống là người dùng, sự kiện và giao dịch đặt vé. Những lớp khác như `Notification`, `Bookmark`, `ChatRoom`, `Message` hay các lớp tiện ích hỗ trợ sẽ được xem là các lớp bổ trợ.

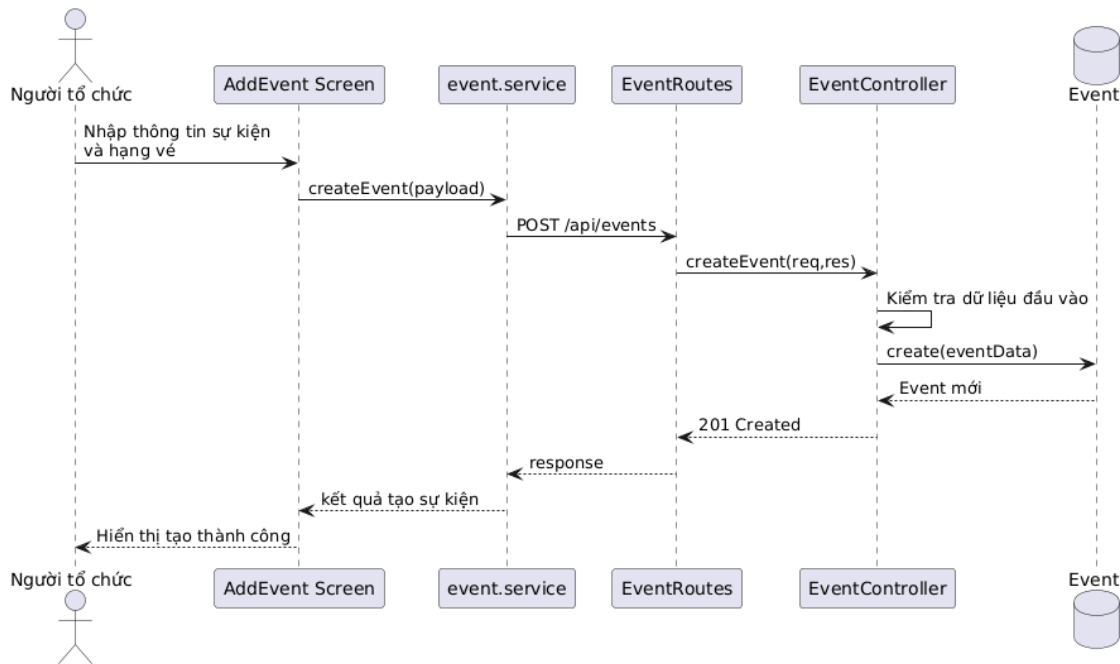
Lớp User

Lớp `User` đại diện cho người sử dụng hệ thống, bao gồm cả người dùng thông thường lẫn quản trị viên. Về mặt thuộc tính, lớp này lưu trữ các thông tin định danh và hồ sơ như `name`, `email`, `password`, `avatar`, `country`, `interests`, `location`, `description`, cùng với các thông tin điều khiển nghiệp vụ như `role`, `verified`, `notificationsEnabled`, `expoPushToken`, `passwordResetCode` và `passwordResetExpires`. Các thuộc tính này cho phép hệ thống không chỉ xác thực người dùng mà còn cá nhân hóa hồ sơ, quản lý quyền truy cập và hỗ trợ các chức năng thông báo đẩy hoặc khôi phục mật khẩu.

Về phương thức, lớp `User` có phương thức quan trọng `matchPassword()` để đối chiếu mật khẩu người dùng nhập vào với mật khẩu đã mã hóa trong cơ sở dữ liệu. Ngoài ra, lớp còn tham gia vào cơ chế tiền xử lý trước khi lưu thông qua thao tác mã hóa mật khẩu bằng `pre-save hook`. Về ý nghĩa thiết kế, `User` là lớp đầu mối cho hầu hết các use case của hệ thống vì mọi nghiệp vụ như đặt vé, tạo sự kiện, nhận thông báo hay nhấn tin đều gắn với một người dùng cụ thể.

Lớp Event

Lớp `Event` đại diện cho một sự kiện được công bố trên hệ thống. Các thuộc tính chính của lớp bao gồm `title`, `description`, `category`, `price`, `date`, `time`, `location`, `coordinates`, `images`, `member`, `attendees`, `rating`, `status`, `approvalStatus`, `organizer` và `ticketTiers`. Trong đó, `organizer` liên kết sự kiện với người tạo sự kiện, còn `ticketTiers` cho phép một sự kiện có nhiều hạng vé khác nhau, mỗi hạng có tên, giá, quota và số lượng đã bán.

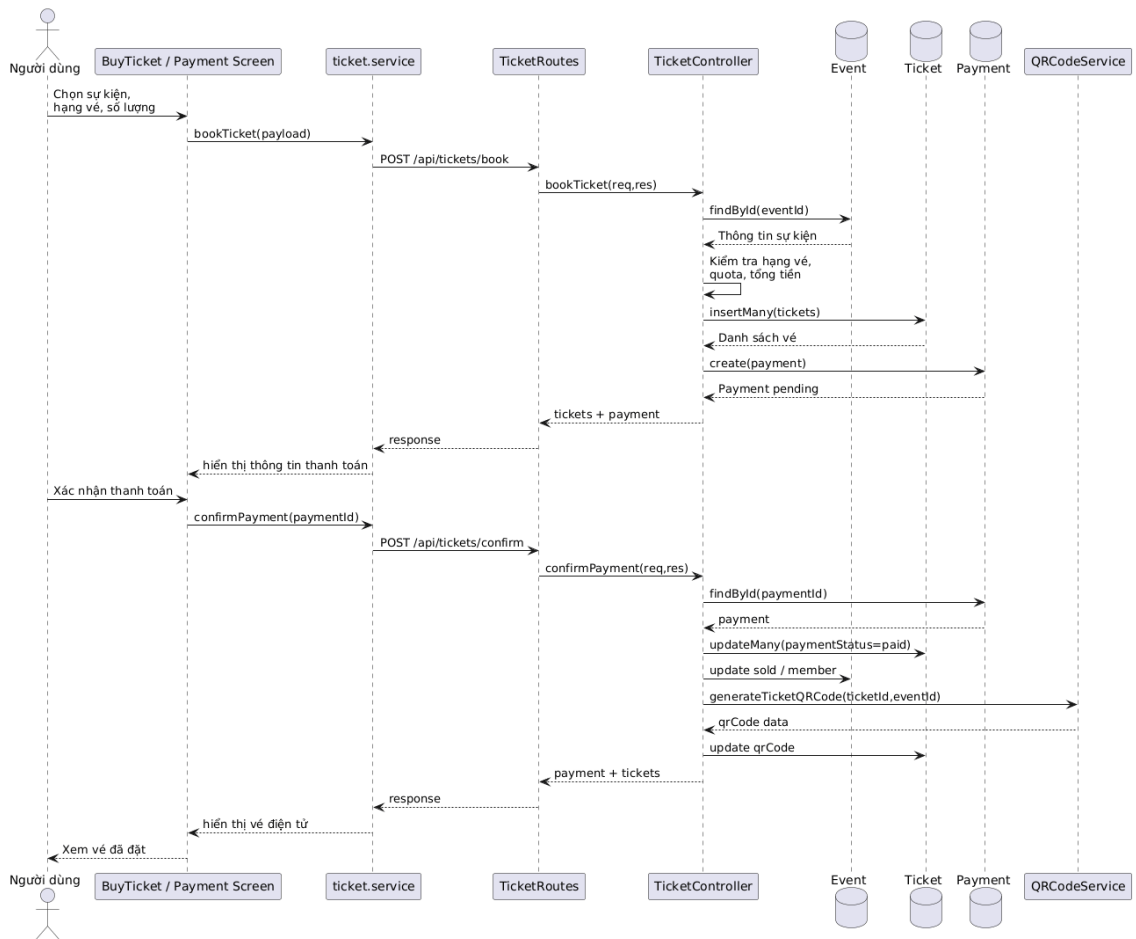


Hình 0.5: Biểu đồ trình tự cho use case Tạo sự kiện

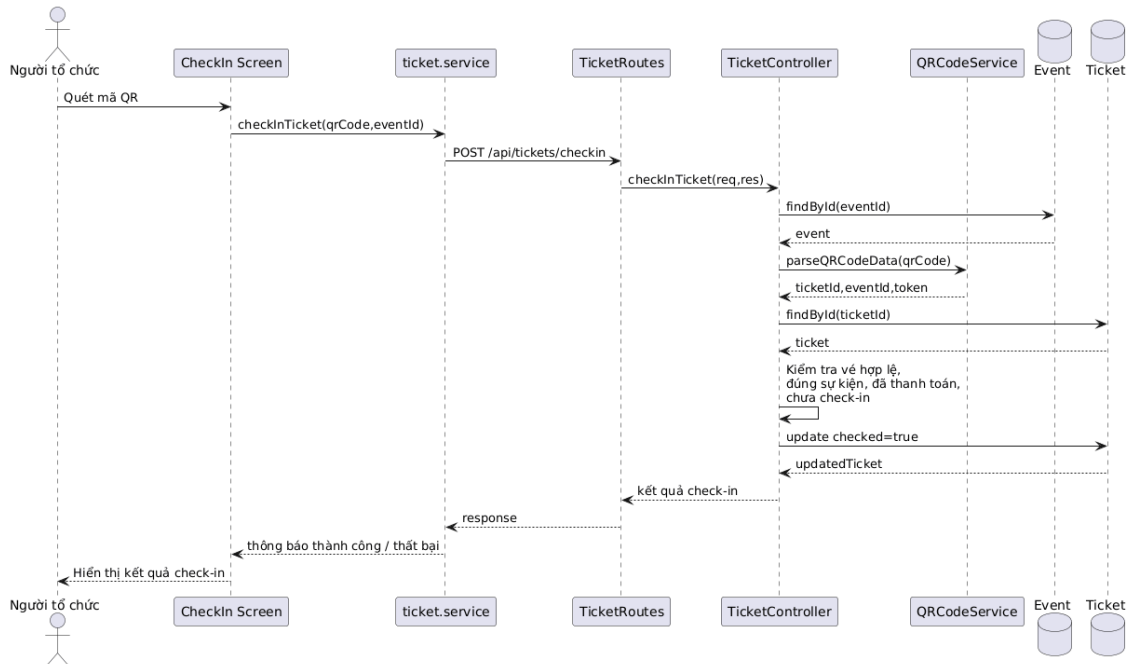
Về mặt phương thức, lớp `Event` không biểu diễn các phương thức nghiệp vụ riêng biệt theo kiểu lớp hướng đối tượng thuần túy, nhưng nó là đối tượng được thao tác xuyên suốt trong các controller như tạo sự kiện, cập nhật sự kiện, duyệt sự kiện, thống kê người tham gia và hiển thị chi tiết sự kiện. Ý nghĩa thiết kế của lớp này nằm ở chỗ nó là trung tâm của phần nghiệp vụ sự kiện: mọi thao tác từ tìm kiếm, đặt vé, thông báo cho tới kiểm soát check-in đều quy chiếu về một đối tượng sự kiện cụ thể.

Lớp Ticket

Lớp `Ticket` đại diện cho vé điện tử của người dùng đối với một sự kiện cụ thể. Các thuộc tính của lớp gồm `user`, `event`, `price`, `ticketType`, `tierName`, `seatInfo`, `qrCode`, `paymentStatus`, `checked`, `checkedAt`, `checkedBy` và `bookedAt`. Nhờ những thuộc tính này, một vé không chỉ là minh chứng cho việc mua thành công, mà còn là thực thể phản ánh trạng thái thanh toán, dữ liệu QR và trạng thái tham dự tại sự kiện.



Hình 0.6: Biểu đồ trình tự cho use case Đặt vé và xác nhận thanh toán



Hình 0.7: Biểu đồ trình tự cho use case Check-in vé bằng mã QR

Ở góc độ nghiệp vụ, lớp `Ticket` tham gia vào các thao tác như khởi tạo vé khi

người dùng đặt chỗ, cập nhật trạng thái thanh toán khi giao dịch được xác nhận, lưu dữ liệu QR để phục vụ việc check-in và cập nhật trạng thái đã sử dụng sau khi quét vé thành công. Đây là lớp có vai trò cầu nối giữa người dùng và sự kiện, bởi nó là thực thể cụ thể hóa mối quan hệ “người dùng tham dự một sự kiện nào đó thông qua một hạng vé xác định”.

Lớp Payment

Lớp Payment biểu diễn giao dịch thanh toán gắn với một hoặc nhiều vé trong hệ thống. Các thuộc tính chính gồm `user`, `event`, `tickets`, `amount`, `method`, `status` và `transactionId`. Cấu trúc này cho phép hệ thống quản lý thanh toán như một thực thể tách biệt thay vì chỉ lưu trực tiếp trạng thái tiền trên từng vé. Điều đó giúp việc truy vết giao dịch, cập nhật trạng thái thanh toán, thống kê doanh thu hoặc tích hợp cổng thanh toán thực tế trong tương lai thuận lợi hơn.

Về phương thức nghiệp vụ, lớp Payment được thao tác chủ yếu trong luồng đặt vé và xác nhận thanh toán. Khi người dùng chọn hạng vé và số lượng vé, hệ thống tạo một giao dịch ở trạng thái chờ xử lý. Sau khi xác nhận thành công, Payment được cập nhật sang trạng thái thành công, đồng thời kéo theo việc cập nhật trạng thái các vé liên quan và số lượng vé đã bán ở sự kiện tương ứng. Vì vậy, lớp này đóng vai trò như một lớp điều phối tài chính, bảo đảm sự nhất quán giữa vé, số lượng vé đã bán và trạng thái giao dịch.

Nhận xét chung về thiết kế lớp

Bốn lớp trên phản ánh lõi nghiệp vụ của hệ thống theo một cấu trúc khá mạch lạc. `User` biểu diễn tác nhân sử dụng hệ thống, `Event` biểu diễn đối tượng trung tâm mà người dùng quan tâm, `Ticket` biểu diễn quyền tham dự của người dùng đối với sự kiện, còn `Payment` biểu diễn giao dịch tài chính đi kèm với việc phát hành vé. Mối liên hệ giữa các lớp này tạo nên phần lớn luồng nghiệp vụ chính trong hệ thống. Việc thiết kế lớp theo hướng tách biệt vai trò như vậy giúp mã nguồn dễ mở rộng hơn, chẳng hạn khi cần bổ sung thêm các chức năng như hủy vé, hoàn tiền, thống kê doanh thu hay xếp chỗ theo khu vực.

0.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB làm hệ quản trị cơ sở dữ liệu, vì vậy mô hình dữ liệu của đề tài được thiết kế theo hướng tài liệu (document-oriented) thay vì quan hệ thuần túy như trong các hệ quản trị SQL. Tuy nhiên, về bản chất nghiệp vụ, các collection trong MongoDB vẫn biểu diễn những thực thể và quan hệ tương tự một mô hình thực thể liên kết. Do đó, việc thiết kế cơ sở dữ liệu vẫn cần bắt đầu

từ việc xác định các thực thể chính, các thuộc tính quan trọng và mối liên hệ giữa chúng. Trên cơ sở đó, hệ thống được tổ chức thành các collection trung tâm như `User`, `Event`, `Ticket`, `Payment`, `Notification`, `Bookmark`, `Reminder`, `Review`, `ChatRoom`, `Message` và `Card`.

Thực thể User

Thực thể `User` lưu thông tin người dùng của hệ thống, bao gồm cả người dùng thông thường và quản trị viên. Các thuộc tính quan trọng của thực thể này gồm họ tên, email, mật khẩu đã mã hóa, avatar, quốc gia, sở thích, vị trí, mô tả hồ sơ, vai trò, trạng thái xác thực, tùy chọn nhận thông báo và token thiết bị phục vụ thông báo đẩy. Đây là thực thể trung tâm của hệ thống vì hầu hết các nghiệp vụ đều quy chiếu về người dùng, từ tạo sự kiện, đặt vé, gửi tin nhắn, nhận thông báo cho đến quản lý bookmark và lời nhắc.

Thực thể Event

Thực thể `Event` mô tả một sự kiện được tạo trong hệ thống. Các thuộc tính chính gồm tiêu đề, mô tả, danh mục, giá mặc định, ngày, thời gian, địa điểm, tọa độ, hình ảnh, số lượng người tham gia, điểm đánh giá, trạng thái sự kiện và trạng thái duyệt. Mỗi sự kiện liên kết với một người tổ chức thông qua thuộc tính `organizer`. Ngoài ra, sự kiện còn chứa một cấu trúc lồng `ticketTiers` để mô tả các hạng vé, trong đó mỗi hạng vé có tên, giá, quota và số lượng vé đã bán. Việc đặt `ticketTiers` bên trong tài liệu sự kiện là một lựa chọn phù hợp với MongoDB vì các hạng vé luôn gắn chặt với một sự kiện cụ thể và thường được truy xuất cùng nhau.

Thực thể Ticket và Payment

Thực thể `Ticket` biểu diễn vé điện tử mà người dùng nhận được khi đặt tham gia một sự kiện. Mỗi vé liên kết với một người dùng và một sự kiện, đồng thời lưu giá vé, loại vé, tên hạng vé, thông tin vị trí ghế nếu có, dữ liệu mã QR, trạng thái thanh toán, trạng thái check-in và thời điểm đặt vé. Đây là thực thể cầu nối giữa người dùng với sự kiện, phản ánh trực tiếp việc một người dùng tham gia một sự kiện nào đó qua một hạng vé xác định.

Thực thể `Payment` dùng để quản lý giao dịch thanh toán. Mỗi thanh toán liên kết với một người dùng, một sự kiện và một tập vé tương ứng. Các thuộc tính chính của nó gồm tổng số tiền, phương thức thanh toán, trạng thái giao dịch và mã giao dịch. Việc tách `Payment` thành một thực thể riêng giúp hệ thống dễ mở rộng sang

các kịch bản thực tế hơn như hoàn tiền, theo dõi giao dịch, thống kê doanh thu hoặc tích hợp cổng thanh toán thật trong tương lai.

Các thực thể hỗ trợ tương tác và theo dõi

Bên cạnh các thực thể cốt lõi, hệ thống còn có nhiều thực thể hỗ trợ cho trải nghiệm người dùng. Thực thể `Bookmark` lưu mối liên hệ giữa người dùng và sự kiện được lưu quan tâm, qua đó hỗ trợ tính năng theo dõi sự kiện. Thực thể `Reminder` lưu các lời nhắc gắn giữa người dùng và sự kiện, giúp hệ thống xác định thời điểm cần gửi nhắc lịch. Thực thể `Review` lưu đánh giá và bình luận của người dùng cho sự kiện, phục vụ tính năng phản hồi và cập nhật điểm đánh giá trung bình. Thực thể `Notification` lưu các thông báo nội bộ mà người dùng nhận được, bao gồm thông báo hệ thống, lời mời, nhắc lịch, thông báo thanh toán hoặc thông báo chat.

Ngoài ra, nhóm chức năng giao tiếp thời gian thực được tổ chức thông qua hai thực thể `ChatRoom` và `Message`. `ChatRoom` lưu thông tin một phòng chat, có thể gắn với một sự kiện và một danh sách thành viên. `Message` lưu các tin nhắn phát sinh trong từng phòng, bao gồm người gửi, nội dung, loại tin nhắn, tệp đính kèm và danh sách người đã đọc. Cuối cùng, thực thể `Card` dùng để lưu thông tin thẻ thanh toán đã lưu của người dùng ở mức tối giản, bao gồm thương hiệu thẻ, bốn số cuối, tháng hết hạn, năm hết hạn và trạng thái thẻ mặc định.

Mối quan hệ giữa các thực thể

Từ góc nhìn E-R, mối quan hệ giữa các thực thể trong hệ thống có thể mô tả như sau. Một `User` có thể tạo nhiều `Event`, trong khi mỗi `Event` chỉ có một người tổ chức chính. Một `User` có thể sở hữu nhiều `Ticket`, còn mỗi `Ticket` chỉ thuộc về một người dùng và một sự kiện. Một `Payment` có thể liên kết với nhiều `Ticket`, nhưng mỗi vé trong ngữ cảnh hệ thống hiện tại chỉ gắn với một giao dịch thanh toán tại một thời điểm. Một `User` có thể tạo nhiều `Bookmark`, `Reminder`, `Review`, `Notification`, `Card` và `Message`. Một `Event` có thể có nhiều `Ticket`, `Review`, `Notification`, `Reminder` và `Bookmark`. Đối với nhóm chat, một `ChatRoom` có thể chứa nhiều `Message` và nhiều thành viên là các `User`, còn mỗi `Message` thuộc về đúng một phòng chat và một người gửi.

Đặc điểm thiết kế dữ liệu trong MongoDB

Do sử dụng MongoDB, các quan hệ trên không được hiện thực bằng khóa ngoại như trong hệ quản trị quan hệ, mà chủ yếu thông qua các trường tham chiếu `ObjectId` và các cấu trúc lồng. Cách làm này giúp hệ thống vừa giữ được tính linh

Nhận xét chung về thiết kế cơ sở dữ liệu

[illegible]

Hình 0.8: Biểu đồ thực thể liên kết của hệ thống event-booking-app

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

17

rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng 1. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Eclipse Oxygen a64 bit	http://www.eclipse.org/
v.v.	v.v.	v.v.

Bảng 1: Danh sách thư viện và công cụ sử dụng

0.3.2 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

0.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

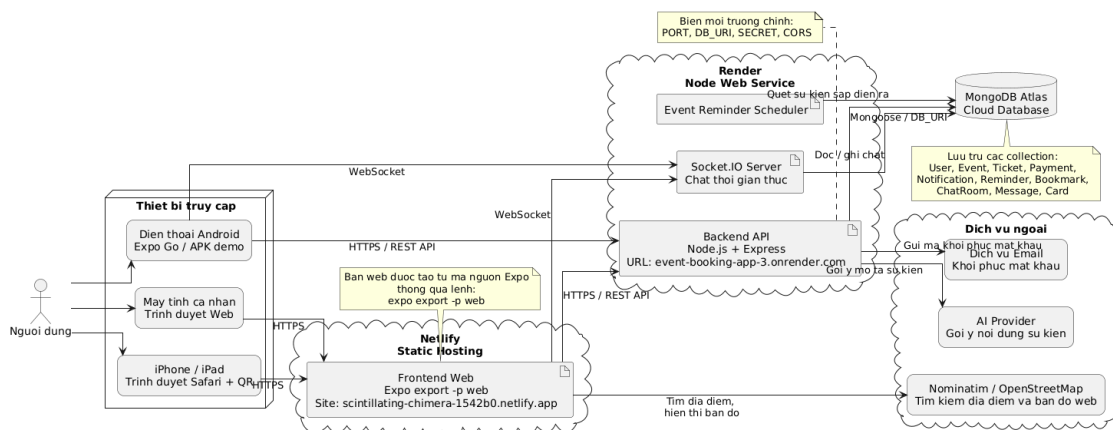
0.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm thử không đạt.

0.5 Triển khai

Trong phạm vi đồ án, hệ thống được triển khai theo mô hình client-server trên môi trường Internet, trong đó frontend đóng vai trò client và backend cung cấp các dịch vụ xử lý nghiệp vụ cũng như truy cập dữ liệu. Mô hình này phù hợp với định hướng mobile-first của hệ thống, bởi các thao tác như đăng nhập, tìm kiếm sự kiện, đặt vé, nhận thông báo, chat và check-in đều được thực hiện ở phía người dùng, sau đó gửi yêu cầu tới máy chủ backend để xử lý. Ở giai đoạn triển khai thử nghiệm, hệ thống được tổ chức thành bốn thành phần chính: frontend Expo/React Native, backend Node.js/Express, cơ sở dữ liệu MongoDB và hạ tầng triển khai trực tuyến cho phép truy cập qua Internet.

Mô hình triển khai thử nghiệm



Hình 0.9: Mô hình triển khai thử nghiệm của hệ thống event-booking-app

Trong quá trình triển khai thực tế, backend được đưa lên dịch vụ Render dưới dạng một *Node Web Service*. Mã nguồn backend được đặt trong thư mục `backend`, được biên dịch từ TypeScript sang JavaScript thông qua lệnh `npm run build`, sau đó khởi động bằng lệnh `npm start`. Tập cấu hình `backend/render.yaml` được sử dụng để mô tả cấu hình triển khai, bao gồm thư mục gốc, lệnh `build`, lệnh `start` và đường dẫn health check là `/api/health`. Việc bổ sung endpoint health check cho phép kiểm tra nhanh trạng thái hoạt động của dịch vụ sau mỗi lần triển khai và hỗ trợ nền tảng Render giám sát tình trạng khởi động của server.

Backend hiện được truy cập qua địa chỉ `https://event-booking-app-3.onrender.com`. Tại phía máy chủ này, ứng dụng Node.js/Express khởi tạo từ tập `backend/src/server.ts`, thực hiện các bước kết nối cơ sở dữ liệu, khởi động bộ lập lịch nhắc sự kiện và khởi tạo Socket.IO cho chức năng chat thời gian thực trước khi mở cổng phục vụ yêu cầu. Cổng mặc định của backend là 5000, tuy nhiên trong môi trường triển khai giá trị cổng được cấp động thông qua biến môi trường `PORT`. Các biến môi trường khác như `DB_URI`, `SECRET`, `JWT_EXPIRES_IN`, `CORS`, `EMAIL_USER`, `EMAIL_PASSWORD` hay các biến liên quan đến AI được cấu hình trong môi trường của Render thay vì ghi cứng trong mã nguồn. Cách làm này giúp hệ thống an toàn hơn và thuận tiện khi thay đổi môi trường chạy.

Cơ sở dữ liệu và kết nối dữ liệu

Ở phía dữ liệu, hệ thống sử dụng MongoDB Atlas và kết nối tới backend thông qua biến môi trường `DB_URI`. Việc đặt cơ sở dữ liệu trên một dịch vụ cloud tách

biệt với backend giúp tách riêng lớp lưu trữ ra khỏi lớp xử lý nghiệp vụ, nhờ đó việc thay đổi máy chủ ứng dụng không làm ảnh hưởng trực tiếp đến dữ liệu. Với mô hình hiện tại, dữ liệu người dùng, sự kiện, vé, thanh toán, thông báo, lời nhắc, bookmark, chatroom và message được lưu dưới dạng các collection riêng biệt. Đây là một lựa chọn phù hợp với MongoDB vì hệ thống có nhiều thực thể nghiệp vụ liên kết với nhau nhưng vẫn đòi hỏi sự linh hoạt khi mở rộng cấu trúc dữ liệu trong các giai đoạn phát triển tiếp theo.

Triển khai frontend phục vụ thử nghiệm

Ở phía client, hệ thống ban đầu được xây dựng như một ứng dụng di động bằng React Native và Expo. Tuy nhiên, trong quá trình thử nghiệm thực tế, việc phân phối ứng dụng lên iOS gặp ràng buộc về tài khoản Apple Developer. Vì vậy, để thuận tiện cho việc trình diễn và kiểm thử trên nhiều thiết bị, đồ án sử dụng thêm một phương án triển khai web từ chính mã nguồn Expo. Cụ thể, frontend được export ra dạng website tĩnh bằng lệnh `expo export -p web`, sinh ra thư mục `frontend/dist`. Thư mục này sau đó được triển khai lên Netlify theo hình thức static hosting và truy cập qua tên miền thử nghiệm `https://scintillating-chimera-1542b0.netlify.app`. Nhờ đó, người dùng có thể mở hệ thống trên iPhone hoặc máy tính bằng trình duyệt thông qua đường link hoặc mã QR mà không cần cài trực tiếp ứng dụng native.

Trong quá trình chuyển frontend sang môi trường web, một số màn hình phụ thuộc thành phần native đã được điều chỉnh để tương thích. Điển hình là màn hình chọn vị trí `SelectLocation`: trên mobile, màn hình này dùng `react-native-maps`, còn trên web hệ thống sử dụng tệp riêng `SelectLocation.web.tsx` với thư viện `Leaflet` và bản đồ `OpenStreetMap` để vẫn bảo đảm người dùng có thể tìm kiếm địa chỉ, xem bản đồ và chọn vị trí tương tự như trên điện thoại. Điều này cho thấy hệ thống không chỉ được triển khai về mặt hạ tầng mà còn được thích nghi ở mức giao diện để phù hợp với môi trường vận hành thực tế.

Liên kết giữa frontend và backend

Sau khi backend đã được triển khai trên Render, frontend được cấu hình sử dụng địa chỉ API công khai `https://event-booking-app-3.onrender.com` trong lớp service giao tiếp. Ở phía backend, biến môi trường `CORS` được đặt theo tên miền frontend trên Netlify để chỉ cho phép client hợp lệ gửi yêu cầu đến server. Nhờ đó, quá trình đăng nhập, lấy danh sách sự kiện, đọc dữ liệu người dùng và các luồng nghiệp vụ chính có thể được kiểm thử trong môi trường thực tế qua Internet, thay vì chỉ chạy trong mạng nội bộ. Việc kiểm tra backend được thực hiện thông

qua endpoint `/api/health`, còn frontend được kiểm tra bằng cách export lại website sau mỗi lần thay đổi cấu hình và triển khai lại thư mục `dist` lên Netlify.

Thiết bị và cấu hình thử nghiệm

Về phía thiết bị người dùng, hệ thống được thử nghiệm trên điện thoại thông minh Android, điện thoại iPhone và trình duyệt web trên máy tính cá nhân có kết nối Internet ổn định. Với Android, ứng dụng có thể chạy qua Expo Go hoặc tiếp tục đóng gói thành bản APK để cài đặt thủ công. Với iOS, phương án trình diễn chính trong đồ án là truy cập bản web triển khai trên Netlify bằng Safari thông qua đường link hoặc mã QR. Về mặt máy chủ, hệ thống không sử dụng một máy vật lý cài đặt thủ công mà tận dụng mô hình *Platform as a Service*: backend chạy trên Render, frontend tĩnh chạy trên Netlify và dữ liệu được lưu trên MongoDB Atlas. Đây là cách triển khai phù hợp với quy mô đồ án vì đơn giản, ít tốn công quản trị hạ tầng nhưng vẫn đủ để tái hiện quá trình vận hành của một hệ thống thực tế có nhiều thành phần phân tán.

Đánh giá mô hình triển khai

Ở mức thử nghiệm, mô hình triển khai hiện tại đã đủ để kiểm tra các luồng nghiệp vụ chính của hệ thống trong môi trường thực tế có kết nối mạng, bao gồm đăng nhập, đọc dữ liệu từ server, tương tác với bản đồ web, quản lý sự kiện và trao đổi dữ liệu qua API. Mô hình này cũng cho phép nhóm phát triển dễ dàng cập nhật backend và frontend một cách độc lập. Tuy nhiên, nếu triển khai ở quy mô lớn hơn trong tương lai, hệ thống vẫn cần được bổ sung thêm các cơ chế như giám sát tài nguyên, lưu log tập trung, tối ưu hiệu năng truy vấn, cấu hình database theo chế độ tin cậy cao, triển khai CDN cho tài nguyên tĩnh và tăng cường bảo mật đối với khóa bí mật, email, token và dữ liệu người dùng.